Alan Bonfatti, Bruno Mansilla, Luciano Castro.

3. Prisma Híbrido

3.1. Explica detalladamente la diferencia conceptual y práctica entre los métodos $queryRaw y $executeRaw en Prisma. ¿Qué tipo de operaciones realizarías con cada uno y qué tipo de valor retorna cada método?

Integramos los datos hechos a la DB con el siguiente comando:

```
npx prisma db execute --config .\prisma7.config.ts --file
.\database.sql
```

La diferencia entre $queryRaw y $executeRaw es que $queryRaw sirve para consultas que devuelven filas como un SELECT, pero devuelve un arreglo con los datos. EJ:

```
const productos = await prisma.$queryRaw`SELECT id, nombre FROM
productos WHERE stock < ${limite}`;
```

Mientras que $executeRaw ejecuta operaciones que no devuelven filas (INSERT, UPDATE o DELETE), devuelve un número con la cantidad de filas afectadas. EJ:

```
    const actualizados = await prisma.$executeRaw`UPDATE productos
SET activo = ${false} WHERE stock = ${0}`;
```

3.2. Prisma utiliza literales de plantilla etiquetados (tagged template literals) cuando escribimos consultas con $queryRaw(ej: prisma.$queryRaw`SELECT * FROM User WHERE id = ${id}`). Explica cómo este mecanismo protege tu aplicación contra ataques de inyección SQL y por qué es peligroso concatenar strings tradicionales en estas consultas.

"id" se considera como parámetro separado de la estructura SQL. La DB recibe la consulta y el valor por separado, en vez de interpretar como parte del código SQL. Si el id contiene un texto malicioso, se tratará como valor y no como una instrucción SQL. Por ejemplo, uso correcto:

```
  const filas = await prisma.$queryRaw`SELECT * FROM productos WHERE id
= ${id}`;
```

Y su uso incorrecto es:

```
const sql = `SELECT * FROM usuarios WHERE email = '${email}'`;
```

3.3. ¿en qué situaciones específicas justificarías abandonar el Prisma Client y escribir una consulta SQL pura en tu aplicación Node.js/JavaScript?

Existen situaciones complejas donde una consulta es difícil o menos eficiente de expresar con Prisma, por ejemplo:
- Informes complejos de muchas uniones, agrupaciones o cálculos.
- Funciones específicas como algunas operaciones en JSON o una búsqueda.
- Consultas analíticas con función de ventana.
- Optimización medida y mejora del rendimiento.

3.4. General en nuestro proyecto con prisma, una ruta que genere un informe que contenga información generada con sql puro

Agregué GET /api/reportes/productos-mas-vendidos, que usa prisma.$queryRaw para contar las unidades vendidas y los pedidos distintos por producto. Excluye los pedidos cancelados y acepta el parámetro opcional limite por defecto 10, con un rango válido de 1 a 100.

```
const rows = await prisma.$queryRaw<ProductoMasVendido[]>`
   SELECT p.id AS producto_id,
          p.nombre AS producto,
          CAST(SUM(dp.cantidad) AS UNSIGNED) AS unidades_vendidas,
          COUNT(DISTINCT pe.id) AS pedidos_distintos
   FROM productos p
   INNER JOIN detalle_pedido dp ON dp.producto_id = p.id
   INNER JOIN pedidos pe ON pe.id = dp.pedido_id
   WHERE pe.estado <> 'cancelado'
   GROUP BY p.id, p.nombre
   ORDER BY unidades_vendidas DESC, p.nombre ASC
   LIMIT ${limite}`;
```